Helix OTA — Stack Research: Eclipse hawkBit

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	Evidence-based evaluation of Eclipse hawkBit as a candidate rollout/campaign server to be wrapped behind the Helix Go control plane. Covers the three APIs (Management API, DDI, DMF), phased rollouts with success/error thresholds, supported databases, containerization, the Go-wrapping strategy, and Android-15 device fit via DDI. All factual claims are sourced; unverifiable specifics are tagged `UNVERIFIED`.
Issues	hawkBit is a JVM/Spring-Boot stack (Java 21) — a heavy runtime relative to the Go control plane; it does not natively understand Android A/B `payload.bin` semantics (treats artifacts as opaque blobs); DMF requires RabbitMQ; the public docs site (`hawkbit.eclipse.dev`) is a client-rendered SPA, so several deep reference pages could not be fetched server-side and were corroborated via the GitHub wiki and the Kynetics static mirror instead.
Issues summary	No blocking defect found for the wrap-and-orchestrate pattern; main concerns are runtime weight, the AMQP dependency for DMF, and Android-payload semantics living in the device client + Helix Go layer rather than in hawkBit.
Fixed	initial research
Fixed summary	

Field	Value
	Primary sources consulted and cross-checked; release/version facts pinned to the GitHub releases page.
Continuation	Feed into ADR-0001 (engine selection, hawkBit vs Mender) and the §11.4.8 deep-web-research decision. If hawkBit is selected, deepen: Management API OpenAPI surface for the Go wrapper, DDI polling/security config for the Android client, and a docker-compose/k8s topology spec.

Table of contents

§1. Summary & confidence

Eclipse hawkBit is a mature, domain-independent OTA back end that provides exactly the rollout/campaign-server capability Helix needs: percentage-based phased rollouts with cascading deployment groups gated by success and error thresholds, plus an emergency-shutdown safety mechanism. It exposes a clean REST **Management API** (control-plane facing), a polling REST **DDI API** (device facing — a natural fit for an Android client), and an AMQP-based **DMF API** (federation/back-end integration). It runs on Spring Boot / Java 21, supports PostgreSQL (and MySQL/MariaDB, H2), and ships as Docker images.

The Helix pattern — a custom Go control plane that *wraps* hawkBit and owns the Helix-specific concerns (Android `payload.bin` packaging, TUF supply-chain signing, Helix telemetry/UX) while delegating rollout orchestration to hawkBit's Management API — is feasible and idiomatic to how hawkBit is designed to be used (it is explicitly a "back end" meant to be driven by 3rd-party apps over the Management API).

Overall confidence: MEDIUM–HIGH. Architecture, API set, rollout semantics, DB support, licensing, and the wrap pattern are well-sourced. Some exact reference-doc strings (precise DDI `_links` field names, default polling interval, exact rollout JSON field names) could not be pulled verbatim from the SPA docs and are tagged `UNVERIFIED — needs confirmation`.

§2. What hawkBit is

> "Eclipse hawkBit is an domain independent back end solution for rolling out software updates to constrained edge devices as well as more powerful controllers and gateways connected to IP based networking infrastructure." — GitHub repo README (`github.com/eclipse-hawkbit/hawkbit`).

- **License:** Eclipse Public License 2.0 (EPL-2.0). (GitHub repo.)
- **Project maturity:** The 1.0 line is described as marking "production readiness, API stability"; project carries "Eclipse Mature" status. (GitHub releases.)
- **Latest release: 1.0.3**, released **2025-04-09** (per the GitHub releases page; the 1.0.x line is 1.0.0 2025-03-31, 1.0.1/1.0.2 2025-04-07, 1.0.3 2025-04-09). `0.10.0` preceded it on 2025-03-16. *(One fetch transcribed the 1.0.3 date as 2026 — the releases listing shows 2025; treating 2025 as authoritative.)*
- **Runtime:** Spring Boot; **Java 21** required. (GitHub repo.)
- **Key modules:** `hawkbit-core`, `hawkbit-repository`, `hawkbit-rest`, `hawkbit-ddi`, `hawkbit-dmf`, `hawkbit-mgmt`, `hawkbit-ui`, `hawkbit-autoconfigure`, `hawkbit-monolith` (the reference Update Server). (GitHub repo.)

Note: hawkBit is OS- and protocol-agnostic on the device side — it does not impose an update mechanism. It manages *which* artifacts go to *which* devices in *what* order and tracks status; the actual install (A/B swap, etc.) is the device client's responsibility. This is precisely the separation Helix wants.

§3. Architecture & the three APIs

The **Update Server** is the central component. It sits behind/between the Management UI + Management API on one side and the device-facing DDI / federation DMF interfaces on the other. (Eclipse newsroom article; concepts docs.)

API	Audience	Transport	Model	Helix role
Management API	Admins / 3rd-party apps (the Helix Go control plane)	REST/HTTPS, JSON + HAL (`application/json`, `application/hal+json`), HATEOAS	Request/ response CRUD + actions	**Primary integration point** — Go control plane drives rollouts here
DDI (Direct Device Integration)	Devices directly	REST/HTTPS, **device polls**	Poll for tasks, POST feedback	**Android client** talks here (or via a thin Helix shim)

API	Audience	Transport	Model	Helix role
DMF (Device Management Federation)	Back-end device-management services	**AMQP / RabbitMQ**	Async push (server→client) + status events	Optional — for a federated/ push integration instead of device polling

Management API

- RESTful CRUD over **targets** (devices) and repository content (software). Base path `…/rest/v1/` (e.g. `http://localhost:8080/rest/v1/`). (Kynetics static mirror of the Management API reference.)
- Supports **filtering, sorting, paging**; **permission-based access control** with standard + custom roles. (Newsroom article; Management API reference.)
- JSON with **HATEOAS** `_links`; content types `application/json` and `application/hal+json`. (Management API reference.)
- Auth: `Authorization` header (basic credentials) in the reference docs; production deployments typically front this with OAuth2/OIDC via Spring Security. *(OAuth2/OIDC specifics UNVERIFIED here — confirm against deployment guide.)*
- Filtering uses an RSQL/FIQL-style query syntax for target filters. *(Exact grammar UNVERIFIED — confirm in reference docs.)*

DDI API (device-facing, polling)

- Devices are identified by **`controllerId`** (not internal target id), so multiple service clients on one physical device can each act as a controller. (DDI wiki.)
- Base poll resource: **`GET /{tenant}/controller/v1/{controllerId}`**, which returns `_links` to sub-resources: **`deploymentBase`**, **`installedBase`**, **`configData`**, **`cancelAction`**. (DDI / Kynetics reference.)
- Device reports progress by POSTing **feedback** with a `status.execution` value. DDI execution states include `DOWNLOAD`, `DOWNLOADED`, `PROCEEDING`, `SCHEDULED`, `RESUMED`, `CANCELED`, `REJECTED`, `CLOSED`; `CLOSED` carries a result of `SUCCESS`/`FAILURE` that the server maps to `FINISHED`/`ERROR` action status. (DDI wiki.)
- **Polling model:** the server tells the device a polling sleep interval (returned in the controller resource / `configData` flow); the device probes on that cadence. (Authentication concepts doc; DDI reference.) *(Exact default interval value UNVERIFIED — confirm `pollingTime` tenant config.)*
- **Authentication options** (Authentication concepts doc + wiki):
  - **TargetToken** — per-device security token; can be retrieved per target via Management API/UI or delivered in the DMF update message.
  - **GatewayToken** — single token valid for all targets of a tenant, sent via HTTP `Authorization: GatewayToken …`; for a trusted gateway/proxy.
  - **Anonymous** — open device access; explicitly "not really sufficient for a production system."
  - **mTLS** — certificate-based mutual TLS, terminated by a reverse proxy in front of hawkBit (no shared token needed).

DMF API (federation, AMQP)

- Built on **AMQP messaging (RabbitMQ 3.6/3.7/3.8)**; uses exchanges/queues/bindings; primary exchange **`dmf.exchange`**; clients set a `reply_to` header. (DMF wiki; GitHub repo.)
- Client→hawkBit messages: `THING_CREATED`, `THING_REMOVED`, `UPDATE_ATTRIBUTES` (modes MERGE/REPLACE/REMOVE), `UPDATE_ACTION_STATUS` (DOWNLOAD/RUNNING/FINISHED/ERROR…), `PING`.
- hawkBit→client messages: `DOWNLOAD_AND_INSTALL` / `DOWNLOAD`, `CANCEL_DOWNLOAD`, `MULTI_ACTION` (when enabled), `THING_DELETED`, `REQUEST_ATTRIBUTES_UPDATE`, `PING_RESPONSE`. (DMF wiki.)
- DMF is a **push** model (server commands targets), the inverse of DDI's device-pull.

§4. Domain model

(Concepts docs / search synthesis; cross-checked against newsroom article.)

- **Artifact** — one file to be rolled out.
- **Software Module** — a collection of artifacts (e.g., an OS module, an app module).
- **Distribution Set** — a collection of software modules; represents the *target state* (the currently-installed software) of a device. By default a device has exactly **one** Distribution Set assigned; **Multi-Assignment** mode relaxes this to allow several simultaneously.
- **Target** — a provisioning target (device), keyed by `controllerId`; has a **Target Type** and attributes.
- **Target Filter** — a saved query selecting a dynamic set of targets; used to feed rollouts and auto-assignments.
- **Rollout** — a managed campaign over a target set, split into ordered **deployment groups** (see §5).

§5. Rollouts: phased deployment with success/error thresholds

This is hawkBit's headline capability and the strongest reason to wrap it. (Rollout Management concepts doc; Rollouts wiki; newsroom article; `ThresholdRolloutGroupErrorCondition.java` in the repo.)

Rollout Management supports (Management API reference / search): - Create, update, and start rollouts. - Select the input target set via **target-filter** functionality. - Select a **Distribution Set** to deploy. - **Auto-split** the input target list into a defined number of **deployment groups** (groups can also be defined explicitly with their own per-group conditions).

Cascading execution between groups is governed by two thresholds: 1. **Success condition (trigger threshold)** — the percentage of targets in the current group that must reach success before the **next** group is started. This is both a safety gate and a

load-distribution mechanism. 2. **Error condition (error threshold)** — an absolute count or percentage of failed installations that, when exceeded, triggers the **error action**.

Error action: the standard/default error action is an **emergency shutdown of the entire rollout** (the rollout is paused/halted so no further groups start). (Rollouts wiki; concepts doc.) An operator can then inspect and resume.

DOWNLOADED state in threshold computation: hawkBit can consider the `DOWNLOADED` state (not only installed) when computing group thresholds — enabling configurations such as "advance to the next group at 50% *updated* OR 80% *downloaded*." (Rollout-management search result; tied to the Separation-of-Download-and-Install feature.) *(Exact configurability surface UNVERIFIED — confirm in current 1.0.x docs.)*

Lifecycle controls: rollouts have **start / pause / resume** controls and an **approval** workflow gate (rollout can require approval before running). Progress is monitored per-group and overall. (Newsroom article; Rollouts wiki.) *(Exact Management API endpoint verbs for start/pause/resume/approve — e.g. `POST …/rollouts/{id}/start` — are UNVERIFIED here; confirm against the OpenAPI/reference before coding the Go wrapper.)*

Dynamic rollouts: newer hawkBit versions add "dynamic" rollouts (a continuously-extending final group that picks up newly-matching targets). *(Presence/behavior in 1.0.x UNVERIFIED — confirm.)*

> Helix mapping: the locked Helix staged-rollout design (5/10/30/…/100 % phases gated by success/error thresholds with pause/halt on breach) maps **directly** onto hawkBit rollout groups + trigger/error thresholds + emergency shutdown. This is a near 1:1 fit and is the core argument for wrapping rather than rebuilding rollout orchestration in Go.

§6. Database support

(GitHub repo README.)

- **Production-grade: PostgreSQL**, **MySQL/MariaDB**. hawkBit ships DDLs for these and provides Docker images bundling the appropriate JDBC drivers.
- **Dev/test: H2** (embedded).
- Microsoft SQL Server: *not listed as production-grade in the current README* — treat as unsupported unless confirmed (`UNVERIFIED`).

> Helix mapping: PostgreSQL is already the chosen Helix relational store, so hawkBit can share the same DB engine family (separate schema/instance recommended to keep Helix control-plane state isolated from hawkBit's internal repository state).

§7. Containerization & deployment topology

(GitHub repo README; newsroom article.)

- Distributed as **Docker images**; the reference **`hawkbit-monolith`** Update Server runs on **port 8080**.
- Two deployment shapes:
  - **Monolith** — single Spring Boot app (Update Server + Management UI + all APIs). Simplest; good for Helix MVP.
  - **Microservices/modular** — the modules can be split (the 2022 "refines software distribution methods" work emphasized a more modular, microservice-friendly structure). Useful for scaling DDI ingress independently from Management.
- **DMF requires RabbitMQ**; DDI + Management do not (RabbitMQ is optional and only needed if DMF is used).
- Persistence requires one of the supported databases (§6); artifact binaries can be stored on the filesystem or an S3-compatible object store via the artifact repository abstraction. *(Exact S3/MinIO support matrix in 1.0.x UNVERIFIED — confirm; Helix design already assumes MinIO/S3 for blobs.)*

> Helix mapping: a docker-compose / k8s topology of `hawkbit-monolith` + PostgreSQL (+ optional RabbitMQ only if DMF is used) + MinIO/S3 for artifacts fits the Helix containerized deployment model. For MVP, monolith + DDI (no RabbitMQ) is the leanest path.

§8. How the Helix Go control plane would wrap hawkBit

hawkBit is explicitly designed to be driven by "3rd party applications" over the **Management API**, so the Helix Go control plane becomes one such application. Recommended division of responsibility:

hawkBit owns (delegated): - Rollout/campaign orchestration: deployment groups, trigger/error thresholds, cascade, emergency shutdown, approval, pause/resume. - Target registry & target-filter evaluation. - Distribution Set / Software Module / Artifact repository and action/status state machine. - DDI device-facing endpoint (poll + feedback) if devices are allowed to talk to hawkBit directly.

Helix Go control plane owns (Helix-specific value): - Helix REST surface (Gin, Brotli, HTTP/3→HTTP/2 fallback) — the public API/UX; translates Helix concepts to Management API calls. - **Android `payload.bin` / `.zip` packaging** and metadata — hawkBit treats these as opaque artifacts; Helix builds them and registers them as Software Modules / Distribution Sets via the Management API. - **TUF supply-chain signing/verification** — sign before upload, embed verification metadata; hawkBit does not provide TUF. - Helix telemetry pipeline, dashboards, and any Helix-native policy not expressible as a hawkBit rollout condition. - Identity/tenancy mapping between Helix users and hawkBit permissions/roles.

Wrapping mechanics (Go): - The Go service calls the Management API over HTTP (standard `net/http` / a typed client generated from hawkBit's OpenAPI). Operations: create/upload Software Module + Artifact → create Distribution Set → create Target Filter → **create Rollout** (set group count, trigger %, error %) → **start**

→ poll rollout/group status → **pause/resume** on Helix policy. - Consume status either by **polling** the Management API (simplest) or by subscribing to hawkBit events. *(Event/webhook push surface for Management-side consumers UNVERIFIED — DMF is AMQP-based and device-oriented; confirm whether a management-event stream exists or if polling is required.)* - Auth Helix→hawkBit: service credentials / OAuth2 against the Management API; Helix never exposes hawkBit directly to the public internet (hawkBit sits behind the Go control plane and/or a reverse proxy).

Net assessment: the wrap is clean because the seam (Management API drives campaigns; DDI serves devices) matches Helix's intended layering. The main custom-code burden stays in Go where Helix wants it (Android payloads, TUF, UX), and the hard, well-tested part (threshold-gated cascading rollouts) is reused rather than reimplemented.

§9. Android 15 device fit via DDI

hawkBit is **OS-agnostic** and does not include an Android-A/B updater; the Android device fit is achieved by a **DDI client on the device** that bridges hawkBit tasks to the Android `update_engine`. (DDI is HTTP-poll + token auth, which is straightforward to implement on Android.)

Fit characteristics: - **Transport:** DDI is plain REST/HTTPS polling — trivially implementable from an Android foreground/background service. `controllerId` = a stable device id; the client polls `GET /{tenant}/controller/v1/{controllerId}`, follows `deploymentBase`, downloads artifacts, and POSTs feedback. - **Auth:** TargetToken per device (recommended) or mTLS via reverse proxy; both are implementable on Android. GatewayToken only if a trusted gateway fronts a fleet. - **A/B install:** hawkBit hands the client artifact URLs + metadata; the **Helix Android client** feeds the `payload.bin` to `update_engine.applyPayload(...)` (Virtual A/B + compression on Android 15) and reports `DOWNLOAD → DOWNLOADED → PROCEEDING → CLOSED(SUCCESS/FAILURE)` back via DDI feedback. The boot-failure auto-rollback is handled by AOSP A/B, **not** by hawkBit — hawkBit only records the resulting action status. - **Separation of download and install:** hawkBit's download/install separation + the `DOWNLOADED` threshold state aligns well with A/B, where staging (write to inactive slot) and activation (reboot) are distinct phases — the client can report `DOWNLOADED` after staging and `CLOSED/SUCCESS` after a successful reboot.

Gaps to own in Helix client/Go layer: - hawkBit has **no Android-payload awareness** — slot semantics, `payload.bin` properties, and post-reboot verification live in the Helix Android client. - hawkBit feedback states are coarser than `update_engine` progress; Helix may want richer telemetry than DDI carries, so a Helix telemetry channel (separate from DDI feedback) is advisable.

> Verdict on Android fit: **good** for the control/orchestration seam (DDI poll + token + feedback is a clean device protocol), with the Android-specific install logic intentionally remaining in the Helix client — consistent with the locked "native A/B + custom control plane" decision.

§10. Pros & cons for Helix

Pros - Threshold-gated, cascading phased rollouts with emergency shutdown are built-in and battle-tested — a near 1:1 match to the locked Helix staged-rollout design. - Three clean APIs with a natural seam: Management API for the Go control plane, DDI for the Android client, DMF optional. - EPL-2.0 (permissive, business-friendly); mature/active project; 1.0 line declared API-stable. - PostgreSQL support aligns with Helix's chosen store; Docker images + monolith/microservice topologies fit Helix's containerized model. - OS-agnostic — does not fight the native-A/B decision; keeps Android specifics in Helix.

Cons - JVM/Spring-Boot/Java-21 runtime — heavier than the Go control plane; adds a second language/runtime to operate. - No native understanding of Android `payload.bin`/A/B semantics or TUF — Helix must own packaging, signing, and slot logic anyway. - DMF (push) requires RabbitMQ; choosing DMF adds an AMQP broker to operate (DDI-only avoids this). - Management-side **event push** for the Go consumer is unconfirmed; may require polling the Management API for rollout/group status. - Docs site is a client-rendered SPA; some exact reference strings are harder to pin (see §11).

§11. Open questions / UNVERIFIED items

These must be confirmed against the live 1.0.x reference docs / OpenAPI before committing in an ADR or coding the Go wrapper:

1. Exact Management API endpoint verbs for rollout **start/pause/resume/approve** and the **create-rollout JSON** field names (group count, trigger/error percentages). `UNVERIFIED`.
2. Whether a **management-side event/webhook stream** exists for the Go control plane, or whether status must be **polled**. `UNVERIFIED`.
3. Default **DDI polling interval** (`pollingTime`) and where it is configured (tenant config). `UNVERIFIED`.
4. Exact DDI `_links` field names and feedback payload schema (verbatim). Partially corroborated (`deploymentBase`, `installedBase`, `configData`, `cancelAction`) but not byte-exact. `UNVERIFIED`.
5. **Dynamic rollouts** availability/behavior in 1.0.x. `UNVERIFIED`.
6. `DOWNLOADED`-state threshold configurability in current docs (the "50% updated OR 80% downloaded" example). `UNVERIFIED`.
7. **Artifact storage backends** in 1.0.x (filesystem vs S3/MinIO support matrix). `UNVERIFIED`.
8. **Microsoft SQL Server** support status (not in README production list). `UNVERIFIED`.
9. Exact **1.0.3 release date** — releases page shows 2025-04-09; one fetch transcribed 2026. Treated as **2025-04-09**; reconfirm. `LOW-RISK / confirm`.
10. RSQL/FIQL target-filter grammar specifics. `UNVERIFIED`.

§12. Sources consulted

Real URLs actually retrieved or searched during this research (June 2026):

- Eclipse hawkBit GitHub repository (README, modules, license, DBs, Java 21, APIs, Docker): https://github.com/eclipse-hawkbit/hawkbit
- GitHub releases (versions & dates, 1.0.x line): https://github.com/eclipse-hawkbit/hawkbit/releases
- DDI API wiki: https://github.com/eclipse-hawkbit/hawkbit/wiki/DDI-API
- DMF API wiki: https://github.com/eclipse-hawkbit/hawkbit/wiki/DMF-API
- Rollouts Management wiki: https://github.com/eclipse-hawkbit/hawkbit/wiki/Rollouts-Management
- Separation of Download and Install wiki: https://github.com/eclipse-hawkbit/hawkbit/wiki/Separation-of-Download-and-Install
- `ThresholdRolloutGroupErrorCondition.java` (error-threshold implementation): https://github.com/eclipse/hawkbit/blob/master/hawkbit-repository/hawkbit-repository-jpa/src/main/java/org/eclipse/hawkbit/repository/jpa/rollout/condition/ThresholdRolloutGroupErrorCondition.java
- Rollout Management concepts (canonical, now redirects to SPA): https://eclipse.dev/hawkbit/concepts/rollout-management/
- Concepts index: https://eclipse.dev/hawkbit/concepts/
- Management API reference (Kynetics static mirror): https://kynetics.github.io/hawkbit/apis/management_api/
- DDI API reference (Kynetics static mirror): https://kynetics.github.io/hawkbit/apis/ddi_api/
- Authentication concepts (TargetToken/GatewayToken/anonymous/mTLS): https://eclipse.dev/hawkbit/concepts/authentication/ and wiki https://github.com/eclipse-hawkbit/hawkbit/wiki/authentication
- Eclipse newsroom: “Eclipse hawkBit Refines Its Software Distribution Methods” (2022): https://newsroom.eclipse.org/eclipse-newsletter/2022/august/eclipse-hawkbit-refines-its-software-distribution-methods
- Project site: https://hawkbit.eclipse.dev/ (client-rendered SPA — limited server-side fetch)

> Note on method: the official docs site moved from `eclipse.dev/hawkbit` / `www.eclipse.org/hawkbit` to a client-rendered SPA at `hawkbit.eclipse.dev`, which does not return prose to a server-side fetcher. Deep reference content was therefore corroborated via the GitHub wiki and the Kynetics static documentation mirror. Items that could not be pinned verbatim are tagged `UNVERIFIED` in §11 rather than guessed.